Hashing

STL containers:

$\text{std::unordered_map}$ (Key-Value Hash Table) is the STL's direct equivalent of a hash table for storing key-value pairs. It provides fast (average $O(1)$) look-up, insertion, and deletion.

$\text{std::unordered_set}$ (Set Hash Table) stores unique elements (only the key)

TLDR:

The core process for insertion is: a key is fed to the hash function to generate a hash code, which is then mapped via the modulo operator to an index (bucket) in the hash table array for storage.

What is Hashing?

Hashing is a technique used to **map data of arbitrary size to data of a fixed size** (the *hash value* or *hash code*). The primary goal is to provide **very fast, $O(1)$ average time complexity**, for operations like searching, insertion, and deletion. This is achieved by using the hash value as an index into an array, often called a **hash table**.

- **Hash Table:** An array structure that stores data based on its hash value.
- **Hash Function:** A mathematical function that converts a given input (key) into a numerical hash code, which is then typically modulo (or remainder) by the size of the array to get a valid index.

What is Collision?

A **collision** occurs when the hash function generates the **same index** (hash value) for **two different input keys**. Since two different pieces of data cannot occupy the same array slot, collision resolution techniques are necessary.

How to Hash?

"How to hash?" refers to the design of the **hash function**. A good hash function should:

1. Be fast to compute.
2. Minimize collisions (distribute keys uniformly across the hash table).
3. Be deterministic (always produce the same hash code for the same input key).

Collision Resolution Techniques

These methods dictate where a new key is placed when its calculated index is already occupied.

Separate Chaining ⛓️

- **Mechanism:** Instead of storing the data directly in the hash table array, each index of the array holds a **pointer to a linked list** (or a dynamic array/vector).
- **Resolution:** When a collision occurs, the new key is simply added to the linked list at that specific index.
- **Pros:** Simple to implement, table never fills up, slower performance degradation.

Open Addressing (Probing)

In Open Addressing, all elements are stored directly within the hash table array itself. When a collision occurs, the algorithm searches for the next available empty slot in the array.

1. **Linear Probing**
    - **Mechanism:** The algorithm checks the next consecutive slot: $H(key), (H(key) + 1) \pmod m, (H(key) + 2) \pmod m$, and so on, where m is the table size.
    - **Problem:** Leads to **Primary Clustering**.
2. **Quadratic Probing**
    - **Mechanism:** The algorithm checks slots by adding squares of the probe count: $H(key), (H(key) + 1^2) \pmod m, (H(key) + 2^2) \pmod m$, etc.
    - **Pros:** Better at reducing primary clustering than linear probing.
    - **Problem:** Leads to **Secondary Clustering**.
3. **Double Hashing**
    - **Mechanism:** A second, different hash function, $H_2(key)$, is used to determine the step size for probing: $H(key), (H(key) + 1 \cdot H_2(key)) \pmod m, (H(key) + 2 \cdot H_2(key)) \pmod m$, etc.
    - **Pros:** Distributes keys most uniformly, significantly reducing both primary and secondary clustering.

Performance and Clustering

Load Factor (α)

The **Load Factor** is a critical metric for a hash table's performance: $$\alpha = \frac{\text{Number of elements stored} (n)}{\text{Size of the hash table} (m)}$$

- **Separate Chaining:** α can be greater than 1 (average length of the linked lists). As α increases, performance degrades toward $O(n)$.
- **Open Addressing:** α **must** be less than 1. As α approaches 1, probing becomes very slow. When α gets too high (typically $0.7 - 0.8$), the table is **resized** (rehashed) to maintain performance.

Primary and Secondary Clustering

These terms describe patterns of occupied slots that severely degrade performance in Open Addressing:

- **Primary Clustering (Linear Probing):** Occurs when a sequence of occupied slots forms a **large contiguous block**. Any new key that hashes to *any* slot within that block will lengthen the entire block, making lookup and insertion operations much slower.
- **Secondary Clustering (Quadratic Probing):** Occurs when multiple different keys that initially hash to the **same starting index** ($H(key)$) follow the exact same sequence of probe slots. While better than primary clustering, it still causes performance degradation.